Informe Técnico — Desarrollo Backend Avanzado: Spring Cloud y Batch

Evaluación Final Transversal (EFT) — Semana 9

Nombre	Nicolás Cavieres
Carrera	Analista Programador
Fecha	09/10/2026
Profesor	Alonso Castillo
Asignatura	Backend III (PBY2203)
Código fuente	https://github.com/nicolasarielcc/EFT_Backend_III

---

1. Resumen

El presente informe documenta la modernización del sistema bancario del Banco XYZ, que migra desde una plataforma legacy (COBOL + scripts Shell sobre mainframe) hacia una arquitectura de microservicios en la nube. La solución implementa la migración de los procesos batch con Spring Batch, el patrón Backend for Frontend (BFF) para tres canales y microservicios resilientes y seguros con Spring Cloud (Config Server, Eureka, Gateway, OAuth 2.0, Resilience4j y Apache Kafka), desplegados con Docker y preparados para AWS.

2. Objetivos y alcance

Reescribir los procesos batch legacy (transacciones, intereses, estados de cuenta) con Spring Batch.
Desacoplar los frontends mediante BFFs por canal.
Dividir el monolito en microservicios resilientes y seguros.
Desplegar la solución en la nube con escalabilidad horizontal.

3. Requerimientos clave del negocio y decisiones de arquitectura

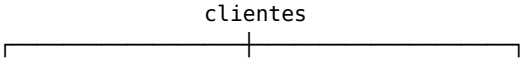
3.1 Requerimientos clave (criterio de la pauta)

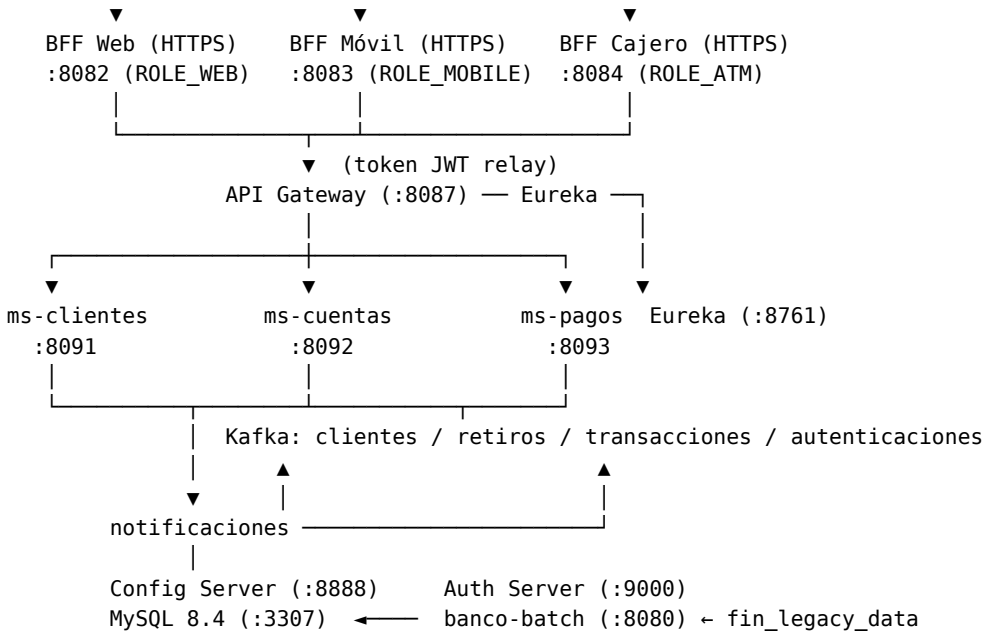
Escalabilidad — el sistema debe crecer horizontalmente ante mayor demanda.
Resiliencia — un fallo en un módulo no debe tumbar todo el sistema.
Seguridad — protección de datos y servicios en un entorno distribuido.

3.2 Justificación de decisiones

Decisión	Justificación
Microservicios en lugar de monolito	Aislamiento de fallos, despliegue independiente y escalado por servicio.
Spring Batch para la migración	Reescribe los jobs legacy con retry/skip, paralelismo y re-ejecución, manteniendo equivalencia de resultados.
BFF por canal	Optimiza respuestas y seguridad por frontend (web completo, móvil ligero, ATM crítico).
Eureka + API Gateway	Detección dinámica de servicios (sin URLs fijas) y enrutamiento/balanceo centralizado.
OAuth 2.0	Autenticación/autorización delegada y distribuida con tokens JWT.
Resilience4j	Circuit Breaker/Retry/Fallback en el consumidor para tolerar fallos de servicios externos.
Kafka	Arquitectura orientada a eventos (desacoplamiento y consistencia eventual).
Docker + AWS	Portabilidad: mismas imágenes en local y nube (RDS, ECS, ALB, MSK).

4. Diagrama de arquitectura





5. Componentes desarrollados

5.1 banco-batch — Migración de procesos batch

Tres jobs Spring Batch sobre fin_legacy_data:

Reporte de Transacciones Diarias — carga movimientos_financieros_diarios.csv, detecta anomalías (montos negativos/cero, fechas inválidas, tipos desconocidos) y genera un resumen.

Cálculo de Intereses — procesa intereses_trimestrales.csv (cuentas de ahorro/préstamo).

Estados de Cuenta Anuales — procesa estados_financieros_anuales.csv.

Cada job usa ItemReader → ItemProcessor → ItemWriter, chunking, RetryPolicy (fallas transitorias de BD) y SkipPolicy (registros inválidos). Es idempotente (limpia sus tablas antes de ejecutar).

5.2 ms-clientes — Gestión de Clientes

CRUD de clientes (información personal y perfiles). Publica eventos clientes en Kafka. Alta/actualización restringida a ROLE_WEB.

5.3 ms-cuentas — Gestión de Cuentas

Apertura, cierre y mantenimiento de cuentas, consulta de saldo y retiros. Publica eventos retiros. Consume transacciones para actualizar saldo (consistencia eventual). Retiro restringido a ROLE_ATM.

5.4 ms-pagos — Procesamiento de Pagos

Procesa pagos, transferencias y depósitos, registra la transacción y publica eventos transacciones (transacciones completadas).

5.5 bff-web, bff-mobile, bff-atm — Backend for Frontend

Web: datos completos + resumen.

Móvil: respuestas ligeras (últimos movimientos).

Cajero: saldo y retiro (operaciones críticas).

Cada BFF expone HTTPS, valida JWT y reenvía el token a los microservicios vía Gateway. Usa Resilience4j (fallback).

5.6 Infraestructura Spring Cloud

config-server: centraliza configuración.

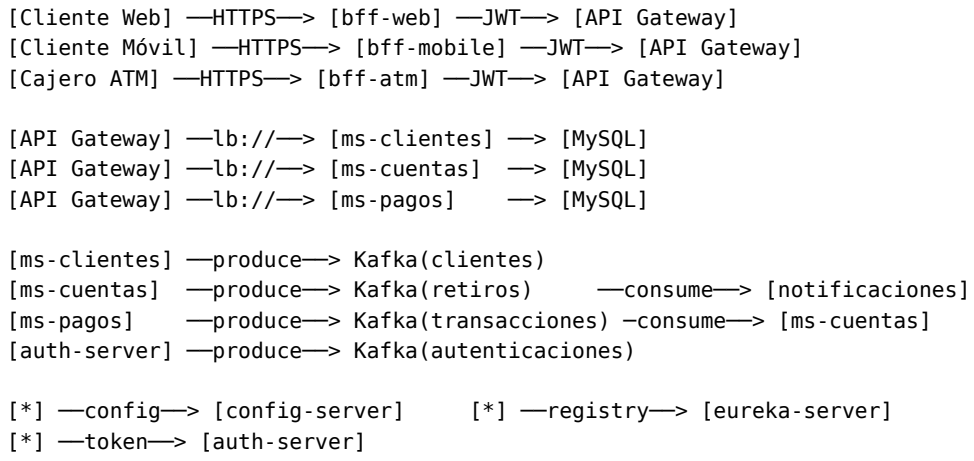
eureka-server: service discovery.

api-gateway: enrutamiento y balanceo de carga (lb://).

auth-server: Authorization Server OAuth 2.0 (JWT RS256).

notificaciones-service: consumidor Kafka (retiros).

6. Diagrama de componentes



7. Casos de uso

Caso de uso	Actor	Flujo
Consultar saldo	Cliente (cajero ATM)	Login (token ATM) → BFF ATM → Gateway → ms-cuentas → saldo
Realizar retiro	Cliente (cajero ATM)	BFF ATM → ms-cuentas → valida saldo → actualiza → evento retiros → notificaciones
Consultar cuenta (detalle)	Cliente Web	Login → BFF Web → Gateway → ms-cuentas + ms-pagos → detalle + resumen
Ver cuentas (ligero)	Cliente Móvil	BFF Móvil → Gateway → ms-cuentas → datos esenciales
Registrar cliente	Empleado (Web)	ms-clientes (ROLE_WEB) → alta → evento clientes
Realizar depósito	Empleado (Web)	ms-pagos → transacción → evento transacciones → ms-cuentas actualiza saldo
Migrar datos legacy	Proceso batch	banco-batch → lee CSV → limpia → MySQL

8. Integración entre componentes

Comunicación síncrona: BFF → API Gateway (HTTP + JWT) → microservicios. El Gateway resuelve los nombres lógicos (lb://ms-*) mediante Eureka.
Comunicación asíncrona: Kafka para eventos (clientes, retiros, transacciones, autenticaciones). ms-cuentas actualiza el saldo consumiendo transacciones (consistencia eventual).
Seguridad: auth-server emite JWT; los microservicios y BFFs son *resource servers* que validan el token vía JWK Set.

9. Seguridad

Autenticación: OAuth 2.0 (client credentials) por canal.
Autorización: roles ROLE_WEB, ROLE_MOBILE, ROLE_ATM; reglas finas por recurso (ej. retiro → ROLE_ATM).
Transporte: HTTPS en los BFFs (certificado autofirmado en desarrollo; ACM en AWS).
Control de acceso: 401 sin token, 403 sin permiso.

10. Resiliencia y mensajería

Resilience4j (en los BFFs, como consumidores): Circuit Breaker (backendCore), Retry y Fallback. Ante la caída de ms-cuentas, el BFF responde con un *fallback* controlado (no rompe el canal).
Kafka: productores en auth-server, ms-clientes, ms-cuentas, ms-pagos; consumidores en notificaciones-service y ms-cuentas.

11. Resultados

Los 3 procesos batch se ejecutan (estado: COMPLETED) con retry/skip; los registros inválidos se

omiten.
Los 3 BFFs autentican por canal y sirven respuestas adaptadas.
Los 3 microservicios se registran en Eureka y son accesibles vía Gateway.
Se verifica 401/403 y el *fallback* de Resilience4j.
Los eventos fluyen productor → consumidor en Kafka.

12. Comparación con el sistema legacy

Aspecto	Legacy	Nuevo sistema
Arquitectura	Monolítica	Microservicios
Escalabilidad	Limitada	Horizontal (Docker/Eureka/ALB)
Tolerancia a fallos	Frágil	Resilience4j + reinicio de contenedores
Batch	Scripts manuales	Spring Batch (retry/skip/paralelismo)
Frontends	Monolito único	BFF por canal
Seguridad	Centralizada	OAuth 2.0 distribuido + HTTPS
Integración	Acoplada	Eventos Kafka

13. Desafíos y soluciones

Desafío	Solución
Datos legacy sucios	Validación/normalización + retry/skip + deduplicación en BD
Anti-patrón de procesadores con estado	Procesadores *stateless*; dedup por clave única
Consistencia distribuida	Eventos Kafka (consistencia eventual)
Fallos de servicios externos	Resilience4j (Circuit Breaker/Retry/Fallback)
Seguridad distribuida	OAuth 2.0 + autorización por canal

14. Propuestas de mejora y próximos pasos

Observabilidad: OpenTelemetry + trazabilidad distribuida (tracing) entre microservicios.
Pruebas: ampliar cobertura de tests de integración y contrato (Consumer-Driven Contracts).
CI/CD: pipeline en GitHub Actions (build, tests, push a ECR, deploy a ECS).
Seguridad: rotación de claves JWT, mTLS entre servicios y secretos en Secrets Manager.
Kafka: patrón Saga (coreografía) para transacciones distribuidas con compensación.
DB por servicio: migrar de BD compartida a esquemas/BBDD independientes por microservicio.

Documento generado para la Evaluación Final Transversal (EFT) — Backend III, Duoc UC.